CS-101 • FOUNDATIONS OF SOFTWARE SYSTEMS & PRODUCT STRATEGY LECTURE 1

System Architecture, Algorithmic Trade-offs & Strategic Alignment

Demonstrating textbook-grade synthesis across Computer Science and Business frameworks

Source Deck: 01-intro-slides.pdf **Date:** Fall 2026
Key Topics: Layered Architecture | Asymptotic Complexity | Binary Search | Strategic Trade-off Matrices

1 Architectural Abstraction & System Layers

Modern software systems manage complexity through **abstraction boundaries**. Rather than reasoning about hardware transistors, operating system kernels, and business logic simultaneously, engineers partition systems into hierarchical layers where each layer provides a well-defined contract to the layer above it.

DEFINITION **Abstraction Barrier**

An **abstraction barrier** is a formal interface separating a high-level consumer from the low-level implementation details of a subsystem. Changes to the internal representation below the barrier do not propagate to consumers above it, provided the interface contract remains invariant.

1. Presentation Layer	Client UI, CLI, and API Gateways handling request validation and serialization.
2. Domain / Business Layer	Core domain invariants, policy rules, workflows, and strategic business logic.
3. Data Access Layer	ORM mappings, query builders, caching strategies (Redis), and transaction coordination.
4. Storage & Infrastructure	Relational/NoSQL storage engines, file systems, and distributed replication logs.

Figure 1.1: Four-tier enterprise software architecture reconstructed natively in Typst.

INTUITION & DEEP DIVE **Why Slides Say 'Decouple Everything' — And What It Actually Means**

Lecture slides often state *"keep layers loosely coupled"* as a single bullet point without explaining the failure mode. Without strict layer boundaries, a schema migration in the storage engine forces edits across UI templates and business rules (**shotgun surgery**). However, excessive indirection also introduces serialization overhead and cognitive load—every abstraction boundary should earn its keep by isolating a dimension that genuinely changes independently.

2 Algorithmic Efficiency & Search

When processing large datasets, the choice of data organization dictates whether latency scales linearly or logarithmically with input size n.

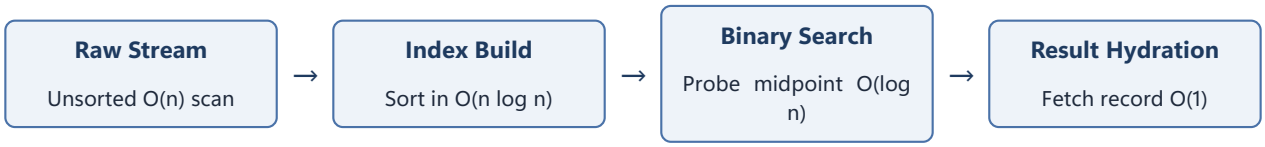


Figure 1.2: End-to-end query execution pipeline from unsorted ingestion to logarithmic lookup.

Algorithm: Iterative Binary Search `Time: O(log n) | Space: O(1)`

Input: Sorted array $A[0 \ldots n-1]$ of comparable elements, target key k.
Output: Index i such that $A[i] = k$, or -1 if $k \notin A$.

```python
def binary_search(arr: list[int], target: int) -> int:
    low, high = 0, len(arr) - 1
    while low <= high:
        # Avoid integer overflow in fixed-width languages:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

PYTHON

EXAMPLE & WALKTHROUGH **Step-by-Step Trace on A = [10, 23, 35, 42, 58, 71, 89], Target k = 58**

1. **Iteration 1:** `low = 0`, `high = 6` $\Longrightarrow$ `mid = 3` ($A[3] = 42$). Since $42 < 58$, discard left half: `low = 4`.
2. **Iteration 2:** `low = 4`, `high = 6` $\Longrightarrow$ `mid = 5` ($A[5] = 71$). Since $71 > 58$, discard right half: `high = 4`.
3. **Iteration 3:** `low = 4`, `high = 4` $\Longrightarrow$ `mid = 4` ($A[4] = 58$). Match found! Return index `4`.

PITFALL / WARNING **Midpoint Overflow & Unsorted Preconditions**

- **Integer Overflow:** Writing `mid = (low + high) / 2` in C, C++, or Java overflows a 32-bit signed integer when `low + high` exceeds $2^{31} - 1$. Always compute `low + (high - low) / 2`.
- **Hidden Sort Cost:** If an array is searched only once, sorting it first ($O(n \log n)$) plus binary search ($O(\log n)$) is **slower** than a single linear scan ($O(n)$). Binary search only amortizes when the dataset is queried repeatedly.

3 Engineering & Business Strategy Alignment

Technical architecture decisions rarely exist in a vacuum; they map directly onto product strategy, unit economics, and time-to-market constraints.

Architecture Style	Initial Velocity	Horizontal Scale	Primary Business Fit
Modular Monolith	High (single repo/ deploy)	Vertical + Read Replicas	Early-to-mid stage product-market fit

Microservices	Low (infra & RPC overhead)	Independent per service	Large orgs needing team autonomy
Serverless / FaaS	Very High (zero ops)	Auto-scaling on demand	Spiky workloads & event-driven pipelines

Figure 1.3: Technical Investment Prioritization Matrix (SWOT / Eisenhower Synthesis)

High Business Value • Low Eng Effort (Quick Wins)

- Add database composite indexes on hot queries.
- Enable HTTP response compression and CDN caching.

High Business Value • High Eng Effort (Strategic Bets)

- Multi-region active-active failover architecture.
- Core domain model refactoring for enterprise multi-tenancy.

Low Business Value • Low Eng Effort (Fill-ins)

- Minor internal CLI ergonomics improvements.
- Non-critical dependency patch upgrades.

Low Business Value • High Eng Effort (Time Sinks)

- Premature migration to Kubernetes before product-market fit.
- Writing a custom ORM or custom serialization protocol.

CASE STUDY **Startup Premature Microservices Migration**

A 6-engineer fintech startup split their MVP into 14 microservices before acquiring 500 active users. Every feature spanning billing and notifications required coordinated deployments across three repositories, distributed tracing debugging, and eventual consistency handling. Consolidating back into a **modular monolith** reduced deployment lead time from 3 days to 25 minutes while preserving clean module boundaries in code.